

Copiar uma árvore binária de busca

Questão 4 · Simulado 2026.2, Questão 02 (Individual, 3,0 pontos)

Enunciado

Implemente uma função recursiva que produza uma cópia completa de uma árvore binária de busca. A nova árvore deve possuir a mesma estrutura e os mesmos valores da original, mas nenhum nó pode ser compartilhado entre as duas árvores.

Assinatura sugerida: `NoArvore* copiarArvore(NoArvore* raiz);`

Trate corretamente a árvore vazia e as falhas de alocação de memória.

Explicação passo a passo

É como tirar uma **fotocópia de um mapa**: o desenho é igual, mas o papel é outro. Se você rabiscar a cópia, o mapa original não muda.

- 1 Árvore vazia?** A cópia também é vazia: devolve NULL. Este é o caso base da recursão.
- 2 Senão,** cria um nó novo com malloc (nada de reaproveitar o nó original) e copia o valor.
- 3** Pede para a própria função copiar a subárvore esquerda e depois a direita. Ela confia que a função sabe fazer o mesmo trabalho para árvores menores.
- 4 Falha de alocação:** se o filho original existia mas a cópia dele veio NULL, algo falhou. Então libera o que já tinha sido copiado e devolve NULL para avisar quem chamou.

A verificação usa a condição "tinha filho e a cópia veio NULL", porque um filho que nunca existiu também resulta em NULL, e isso não é erro.

Código completo

Arquivo: 04_copiar_arvore.c. Para compilar: `gcc -Wall -o programa`

`04_copiar_arvore.c && ./programa`

```
/* PDF - Questão 02: copiar árvore binária de busca (recursivo) */
#include <stdio.h>
#include <stdlib.h>

typedef struct NoArvore {
    int valor;
    struct NoArvore* esq;
    struct NoArvore* dir;
} NoArvore;

NoArvore* inserir(NoArvore* raiz, int v) {
    if (!raiz) {
        NoArvore* n = (NoArvore*) malloc(sizeof(NoArvore));
        if (!n) return NULL;
        n->valor = v; n->esq = n->dir = NULL;
        return n;
    }
    if (v < raiz->valor) raiz->esq = inserir(raiz->esq, v);
    else raiz->dir = inserir(raiz->dir, v);
    return raiz;
}

void liberarArvore(NoArvore* raiz) {
```

```

    if (!raiz) return;
    liberarArvore(raiz->esq);
    liberarArvore(raiz->dir);
    free(raiz);
}

NoArvore* copiarArvore(NoArvore* raiz) {
    /* Caso base: árvore vazia -> a cópia também é vazia */
    if (raiz == NULL) return NULL;

    /* Cria um nó NOVO (nada de compartilhar com a original) */
    NoArvore* novo = (NoArvore*) malloc(sizeof(NoArvore));
    if (novo == NULL) return NULL; /* falha de alocação */

    novo->valor = raiz->valor;

    /* Copia o lado esquerdo */
    novo->esq = copiarArvore(raiz->esq);
    if (raiz->esq != NULL && novo->esq == NULL) { /* tinha filho, mas a cópia falhou */
        free(novo);
        return NULL;
    }

    /* Copia o lado direito */
    novo->dir = copiarArvore(raiz->dir);
    if (raiz->dir != NULL && novo->dir == NULL) {
        liberarArvore(novo->esq); /* desfaz o que já foi copiado */
        free(novo);
        return NULL;
    }
    return novo;
}

void emOrdem(NoArvore* r) {
    if (!r) return;
    emOrdem(r->esq); printf("%d ", r->valor); emOrdem(r->dir);
}

int main(void) {
    int v[] = {50, 30, 70, 20, 40, 60, 80};
    NoArvore* orig = NULL;
    for (int i = 0; i < 7; i++) orig = inserir(orig, v[i]);

    NoArvore* copia = copiarArvore(orig);

    printf("Original: "); emOrdem(orig); printf("\n");
    printf("Copia: "); emOrdem(copia); printf("\n");
    printf("Raízes em endereços diferentes? %s\n", (orig != copia) ? "sim" : "nao");

    /* Teste de independência: mexer na cópia não afeta a original */
    copia->valor = 999;
    printf("Depois de mudar a raiz da copia -> original: %d | copia: %d\n",
           orig->valor, copia->valor);

    printf("Arvore vazia: %s\n", copiarArvore(NULL) == NULL ? "ok (NULL)" : "erro");

    liberarArvore(orig); liberarArvore(copia);
    return 0;
}

```

Teste executado

Árvore com 50, 30, 70, 20, 40, 60, 80. O teste confirma que os valores são iguais, que as raízes têm endereços diferentes e que mudar a cópia não afeta a original.

```

Original: 20 30 40 50 60 70 80
Copia:    20 30 40 50 60 70 80
Raízes em endereços diferentes? sim
Depois de mudar a raiz da copia -> original: 50 | copia: 999
Arvore vazia: ok (NULL)

```